# C++ 指针加减运算规则（结合数组，一看就懂）

指针加减不是简单的「地址数字加1减1」，而是**以「指针指向类型的大小」为单位进行偏移**，这是最核心的规则！

## 一、核心本质：单位是「类型大小」，不是「字节」

### 规则公式

对于指针 `T* p`（`T` 是数据类型，比如 `int`、`char`）：

- `p + i` → 地址向后偏移 `i × sizeof(T)` 个字节
- `p - i` → 地址向前偏移 `i × sizeof(T)` 个字节
- 结果：指针指向「当前位置的第 `i` 个同类型元素」

### 通俗比喻

把指针看作「酒店房间的门牌号」：

- `int* p` → 每个房间住 4 个人（`sizeof(int)=4` 字节）
- `p + 1` → 不是门牌号+1，而是**走到下一个房间**（门牌号+4）
- `char* p` → 每个房间住 1 个人（`sizeof(char)=1` 字节）
- `p + 1` → 门牌号+1，走到下一个房间

## 二、代码直观验证（不同类型对比）

### 1. int 类型指针（占 4 字节）

Code block

```
#include <iostream>
using namespace std;

int main() {
    int arr[3] = {10, 20, 30};
    int* p = &arr[0]; // p 指向 arr[0]

    cout << "p 指向的地址: " << p << endl;    // 比如 0x62ff00
```

```
    cout << "p + 1 指向的地址: " << p + 1 << endl; // 0x62ff04 (+4字节)
    cout << "p + 2 指向的地址: " << p + 2 << endl; // 0x62ff08 (+8字节)

    cout << "*(p + 1) = " << *(p + 1) << endl; // 输出 20 (等价 arr[1])
    cout << "*(p + 2) = " << *(p + 2) << endl; // 输出 30 (等价 arr[2])
    return 0;
}
```

### 2. char 类型指针（占 1 字节）

Code block

```
char str[] = "abc";
char* p = &str[0];

cout << "p 指向的地址: " << (void*)p << endl;    // 比如 0x62ff00
cout << "p + 1 指向的地址: " << (void*)(p + 1) << endl; // 0x62ff01 (+1字节)
cout << "*(p + 1) = " << *(p + 1) << endl; // 输出 'b'
```

## 三、指针减法：两个指针相减的含义

### 规则

只有**同类型、指向同一块连续内存**（比如同一个数组）的两个指针才能相减：

- `p2 - p1` → 结果是「两个指针之间的**元素个数**」，不是字节数
- 公式：`(地址差) / sizeof(T)`

### 示例

Code block

```
int arr[5] = {1,2,3,4,5};
int* p1 = &arr[1]; // 指向第2个元素
int* p2 = &arr[4]; // 指向第5个元素

cout << "p2 - p1 = " << p2 - p1 << endl; // 输出 3 (不是 12 字节)
// 地址差: (arr[4]地址 - arr[1]地址) = 3×4=12字节 → 12 / 4 = 3 个元素
```

## 四、指针加减的核心限制

1. **只能对同类型指针运算**
   - 不能把 `int*` 和 `char*` 加减，编译器会报错
2. **不能越界访问**
   - 指针加减后必须仍在原数组/内存块范围内，否则是未定义行为（程序崩溃/乱码）

Code block

```
int arr[3] = {1,2,3};
int* p = arr;
p + 3; // ✅ 合法（指向数组末尾后一位，可用于判断边界）
p + 4; // ❌ 越界，危险！
```

3. **不能对空指针/野指针运算**
   - `nullptr + 1` 或未初始化指针加减，会导致程序崩溃
4. **指针没有乘法/除法**
   - 只有加减运算，没有 `p * 2` 或 `p / 2` 这种写法

## 五、和数组的联动：指针遍历数组

因为数组名是「首元素常量指针」，所以用指针加减可以完美遍历数组：

Code block

```
int arr[4] = {10,20,30,40};
int* p = arr;

for (int i = 0; i < 4; i++) {
    cout << *(p + i) << " "; // 依次输出 10 20 30 40
}
// 等价于 arr[i]
```

## 六、一句话总结

✅ **指针加减 = 按「指向类型的大小」为单位，在内存中前后移动元素**

✅ `p + i` → 第 `i` 个同类型元素

✅ `p2 - p1` → 两个指针之间的元素个数

✅ 永远记住：单位是「元素个数」，不是「字节数」！

要不要我给你出一道**指针加减运算的练习题**，帮你巩固这个知识点？

（注：文档部分内容可能由 AI 生成）